

Database Design Entity-Relationship (ER) Modeling

COSC 304 – Introduction to Database Systems



Database Design

The ability to design databases and associated applications is critical to the success of the modern enterprise.

Database design requires understanding both the operational and business requirements of an organization as well as the ability to model and realize those requirements in a database.

Developing database and information systems is performed using a *development lifecycle*, which consists of a series of steps.

Design documents help developers, analysts, and database administrators understand the structure, meaning, and intended use of the data.

The Importance of Database Design

Software and IT projects remain challenging:

- Large software projects have averaged 66% over budget and 33% over schedule.
- Across IT projects more broadly, average overruns have been 75% in cost and 46% in schedule, with 39% less value delivered than predicted.
- Recent project-management research finds only about half of software-development projects are clearly successful, with roughly 1 in 10 classified as outright failures.

Have you been on a project that **failed**? A) Yes B) No

Common contributors to project problems include unclear requirements, poor planning and design, technical complexity, and organizational issues.

- Sources: McKinsey/Oxford IT project studies; PMI project success research.

Software and System Development

Software development follows an iterative process with steps:

- **Specification** – capturing user and system requirements, goals, and timelines.
- **Design** – develop models for system architecture, data, and behavior. Design the database and associated application, including transactions and user interfaces.
 - Recommend: Solid DB design before prototyping application.
- **Implementation** – create database structures, queries, and application code.
- **Testing** - executing programs to determine errors and issues.
- **Maintenance** – monitoring and maintaining the system after installation.
 - DBMS maintenance includes monitoring performance, security, and upgrades.

Specification

Specification involves:

- project purpose, goals, success criteria, timelines, and deliverables
- users, requirements, and existing organizational processes
- development standards and integration with other systems
- privacy, security, legal, and regulatory requirements

Specification: Mission Statements

The **mission statement** defines the major project objective and provides a clear basis for determining success.

NASA's mission statement when going to the moon:

"I believe that this nation should commit itself to achieving the goal, before this decade is out, of landing a man on the moon and returning him safely to Earth." (John F. Kennedy May 25, 1961)

- The objective was clear and measurable: reach the Moon and return safely before the end of the decade.
- NASA fulfilled that goal on July 20, 1969, when Apollo 11's lunar module *Eagle* touched down in the Sea of Tranquility, with Neil Armstrong and Buzz Aldrin aboard.

Specification: The "Project Champion"

A **project sponsor** or **champion** is someone who supports the project, helps resolve organizational barriers, and takes ownership of its success.

Projects are more difficult to deliver when there is no clear organizational ownership.

- External projects still need an internal sponsor.
- Internal projects need someone who can help resolve competing priorities, resources, and adoption issues.

Bottom line: Successful projects need clear ownership and organizational support.

Specification: Requirements Gathering

Requirements gathering collects details on organizational processes and user issues.

- Often this information is not fully documented and must be discovered from users, existing systems, processes, and data.
- Methods include user interviews, observation, document/system review, and analysis of existing data.
 - Who are the users? Group them into classes.
 - What do they do now? (existing systems/processes)
 - What problems or opportunities exist?
- Determine the data used by the organization, identify data relationships, and determine how data is used and generated.
 - Identify unique fields (keys)
 - Determine data dependencies, relationships, and constraints (high-level)
 - Estimate the data sizes and their growth rates



Database Design



Database design is divided into three phases:

- **Conceptual database design** - constructs a high-level conceptual data model of the domain, independent of a particular implementation data model or DBMS.
 - Specifies entities, attributes, relationships, and constraints.
- **Logical database design** - constructs a logical data model of the domain using a particular implementation data model, but independent of a particular DBMS.
 - Commonly use the relational model but other models include document, graph, and object-oriented models.
 - The features of the chosen data model can now be used (e.g., relations, primary keys, and foreign keys in the relational model).
- **Physical database design** – determines how the logical data model is implemented for a particular DBMS.
 - Includes DBMS-specific tables/data types and constraints, indexes, storage structures, partitioning, security, and other implementation choices.
 - Physical database design is **how**, and logical database design is the **what**.

Database People: DA and DBA

Database roles:

- **Database administrator (DBA)** - deploys, configures, secures, monitors, and maintains database systems.
- **Data administrator (DA)** focuses on organizational data policies, governance, standards, and planning.
- A **data architect** selects systems to use, makes design decisions, and evaluates current and future systems at the architectural level.

These responsibilities may be distributed among DBAs, data architects, data engineers, security specialists, and governance teams in modern organizations.

The DBA is commonly involved in physical design and management of production systems.

Entity-Relationship Modeling

Entity-relationship modeling is a top-down approach to conceptual database design that models the data as entities, attributes, and relationships.

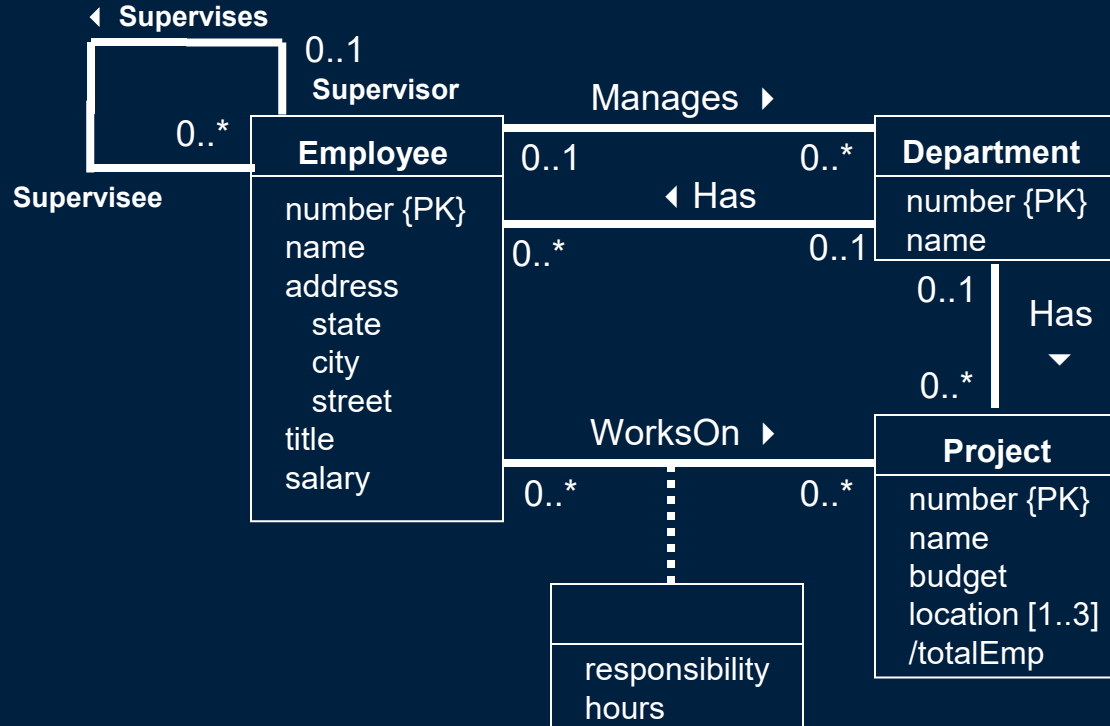
- The entity-relationship (ER) model was proposed by Peter Chen in 1976. We will represent ER models using UML-style notation.

The ER model refines entities and relationships by including properties of entities and relationships called *attributes*, and by defining *constraints* on entities, relationships, and attributes.

The ER model conveys knowledge at a high-level (conceptual level) which is suitable for interaction with technical and non-technical users.

Since the conceptual ER design is independent of a particular DBMS and implementation model, it can later be mapped to a logical model such as the relational model.

ER Model Example in UML notation



Entity Types

An **entity type** is a group of objects with the same properties which are identified as having an independent existence.

- An entity type does not always have to be a physical real-world object such as a person or department. It can be an abstract concept such as a project or job.

An **entity instance** is a particular example or occurrence of an entity type.

- For example, an entity type is Employee. An entity instance is 'E1 - John Doe'.

Representing Entity Types

Entity types are represented by rectangles with the name of the entity type in the rectangle.

Examples:



- An entity type name is normally a singular noun.
 - That is, use Person instead of People, Project instead of Projects, etc.
- The first letter of each word in the entity name is capitalized by convention.

Entities Question

Question: How many of the following statements are **true**?

- 1) Entity types are represented using a rectangle box.
- 2) An entity is always a physical object.
- 3) An entity type is named using a plural noun.
- 4) Employee number is an entity.

A) 0 B) 1 C) 2 D) 3 E) 4

Relationships

A **relationship type** is a set of associations among entity types. Each relationship type has a name that describes it.

A **relationship instance** is a particular occurrence of a relationship type that relates entity instances.

There can be more than one relationship between two entity types.

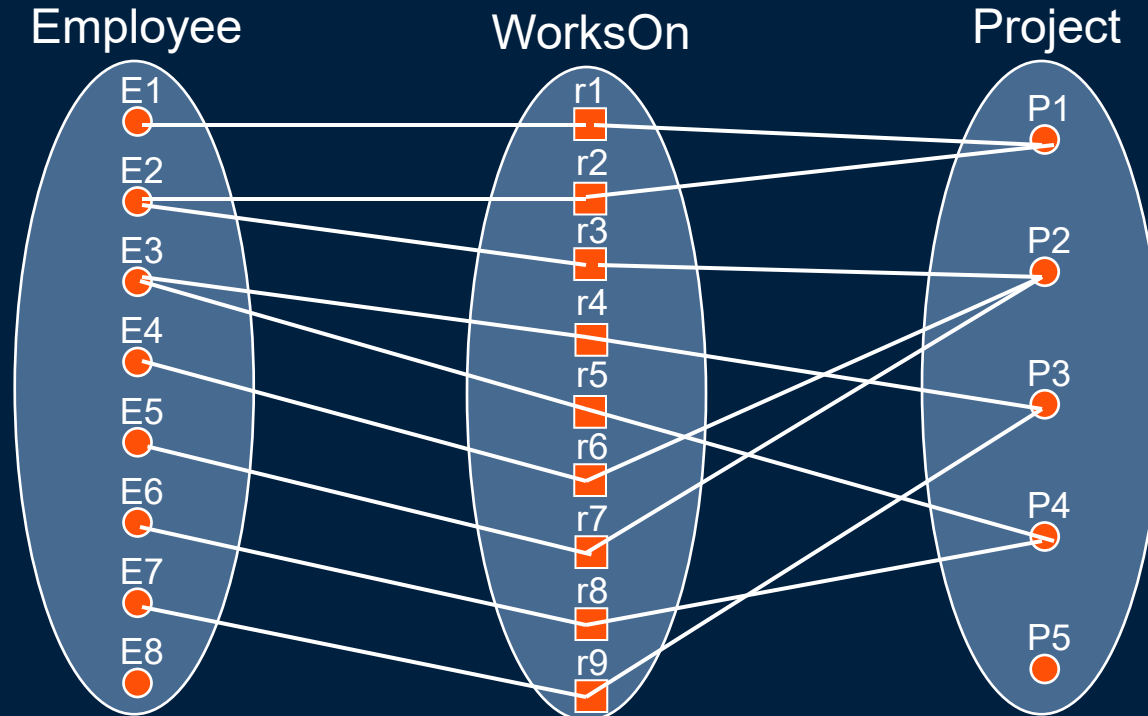
Representing Relationship Types

The relationship type is represented as a labeled edge between the two entity types.



- A relationship type name is normally a verb or verb phrase. A small arrow beside the relationship name indicates the direction in which the name should be read.
- The first letter of each word in the name is capitalized.
- **Do not put arrows on either end of the line.**

Visualizing Relationships



Note: This is an example of a many-to-many relationship. A project can have more than one employee, and an employee can work on more than one project.

Relationship Degree

The **degree of a relationship type** is the number of entity-type participations (roles) in the relationship.

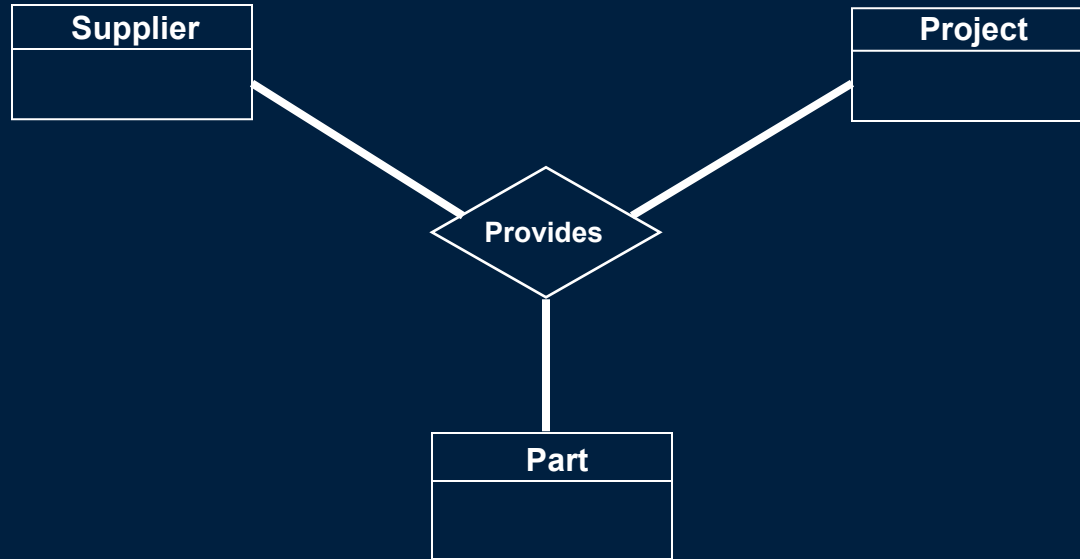
- For example, WorksOn is a relationship type of degree two as the two participating entity types are Employee and Project.
 - Note: This is not the same as degree of a relation which was the number of attributes in a relation.
- An entity type may participate more than once in different roles.

Relationships of degree two are *binary*, of degree three are *ternary*, and of degree four are *quaternary*.

- Relationships of arbitrary degree N are called *n-ary*.

Use a diamond to represent relationships of degree higher than two.

Ternary Relationship Type Example



A project may require a part from multiple different suppliers.

- For example, an employee has a supervisor. The supervisor is also an employee. Each *role* has a *role name*.

```

graph TD
    Supervisor[Supervisor] -- Supervises --> Supervisee[Supervisee]
  
```

- 21

Relationship Question

Question: How many of the following statements are **true**?

- 1) Relationships are represented using a directed edge (with arrows).
- 2) A relationship is typically named using a verb.
- 3) It is not possible to have a relationship of degree 1.
- 4) The degree of a relationship is the number of attributes it has.
- 5) A diamond is used to represent a relationship of degree larger than two.

A) 0 B) 1 C) 2 D) 3 E) 4

Attributes

An **attribute** is a property of an entity type or a relationship type.

- For example, entity type Employee has attributes name, salary, title, etc.

Some rules:

- By convention, attribute names begin with a lower case letter.
- Attribute names are typically adjectives or nouns.
- Each attribute has a *domain*, which is the set of allowable values for the attribute.
- An attribute may be single valued or multi-valued.
- An attribute may be simple if it contains a single component (e.g. salary) or composite if it contains multiple components (e.g. address).
 - Question: Is the name attribute of Employee simple or composite?
- A **derived attribute** is an attribute whose value is calculated from other attributes but is not physically stored.

Representing Attributes

In UML attributes are listed in the rectangle for their entity. Tags are used to denote any special features of the attributes.

- multi-valued attribute: *attributeName* [*minVals*..*maxVals*]
 - e.g. `phoneNumber [1..3]`
- derived attribute: */attributeName* (e.g. */totalEmp*)
 - Derived attribute is calculated from other data rather than being stored in the database.
- partial primary key: {PPK} – for key field of weak entity
 - A **weak entity type** is an entity type whose existence depends on another entity type.

Attribute Question

Question: How many of the following statements are **true**?

- 1) Attributes are properties of either entities or relationships.
- 2) An attribute may be multi-valued.
- 3) A composite attribute contains two or more components.
- 4) Each attribute has a domain representing its allowable values.
- 5) Attribute names are typically verbs.

A) 0 B) 1 C) 2 D) 3 E) 4

Representing Attributes and Keys

A **candidate key** is a minimal set of attributes that uniquely identifies each instance of an entity type.

A **primary key** is a candidate key that is selected by the designer to identify each instance of an entity type.

- Primary key attributes are labeled with { PK } in diagram.
- Note: No foreign keys in ER model but may see { FK } notation in logical diagram.

A **composite key** is a key that consists of two or more attributes.

Key Question

Question: How many of the following statements are **true**?

- 1) It is possible to have two candidate keys with different numbers of attributes.
- 2) A composite key has more than 1 attribute.
- 3) The computer picks the primary key used in the design.
- 4) A relationship has a primary key.
- 5) An attribute has a primary key.

A) 0 B) 1 C) 2 D) 3 E) 4

Attributes on Relationships

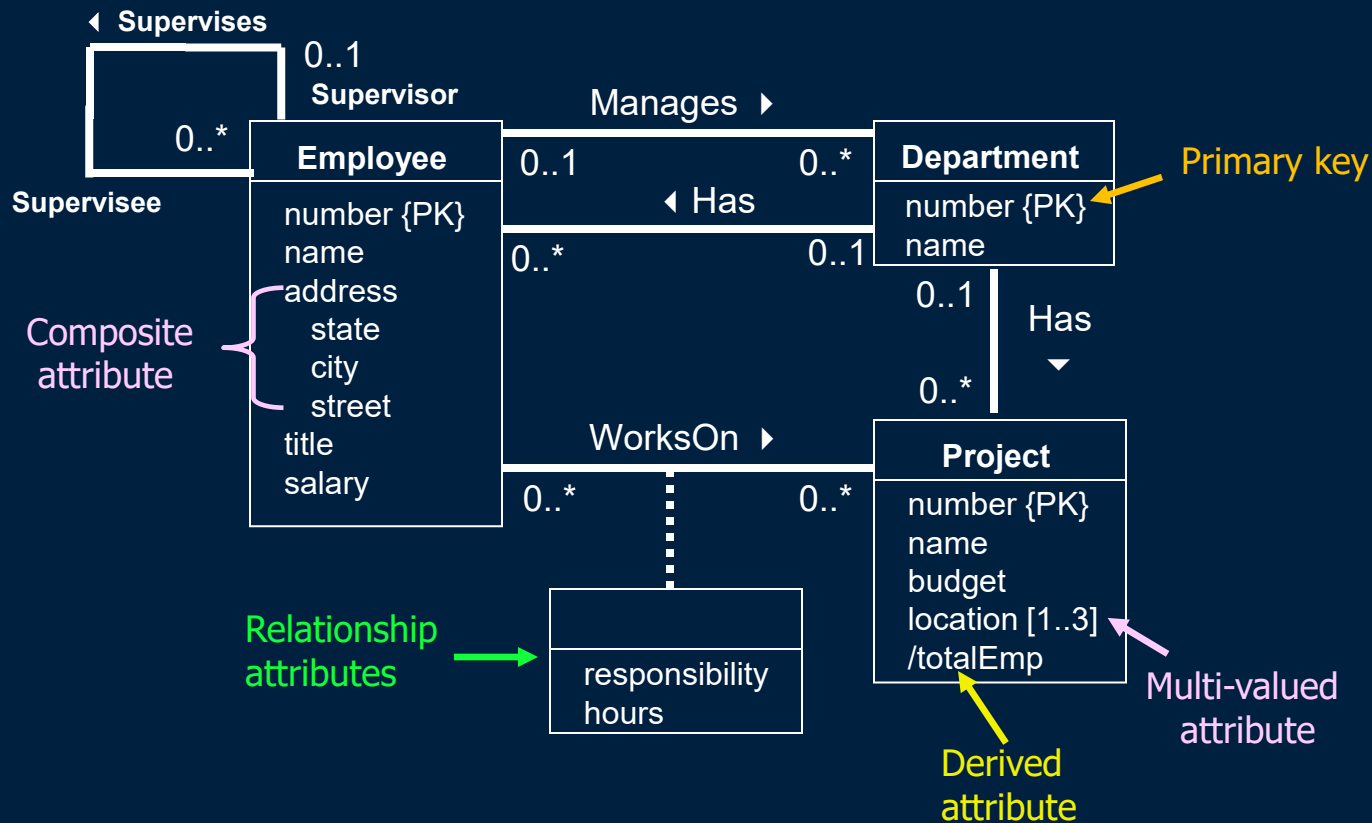
An attribute may be associated with a relationship type.

For example, the WorksOn relationship type has two attributes: responsibility and hours.

These attributes belong to the relationship because their values depend on a particular Employee–Project pairing.

Relationship attributes are represented as a separate box connected to the relationship using a dotted line.

Attributes in UML Notation



ER Design Question #1

Construct an ER model for a university database where:

- Each student has an id, name, gender, birthdate, and GPA.
- Each professor has a name and is in a department. Assume professor name is a primary key.
- Each department offers courses and has professors. A department has a name and a building location.
- Each course has a name and number and may have multiple sections.
- Each section is taught by a professor and has a section number. A section number is unique only within its course.
- Students enroll in sections of courses and receive a grade.

Conclusion

Database design is divided into three phases:

- Conceptual database design
- Logical database design
- Physical database design

Effective requirements gathering is an important skill to master. Projects benefit from clear goals, success criteria, requirements, and organizational ownership.

ER (conceptual) design is performed at a high level of abstraction involving entities, relationships, and attributes.

- There are several ER diagram notations. We use UML-style notation.

Objectives

- Describe the three phases of database design and the result of each phase.
- Describe conceptual, logical, and physical data models.
- Describe how the roles of DBA and DA fit in database design.
- Define and identify entity types, relationship types, relationship degree, recursive relationships, and attributes in an ER model.
- Identify primary keys, composite attributes, multivalued attributes, derived attributes, and relationship attributes.

Construct a basic ER model from a simple domain description using UML-style notation. ★



THE UNIVERSITY OF BRITISH COLUMBIA

